

AGENCYOS

Database & Domain Model

Document 21 of the AgencyOS Master Documentation Set

ENTITIES • RELATIONSHIPS • STATES • EVENTS • TABLES • INVARIANTS • VERSIONING

Purpose: Define the canonical data and domain model that connects AgencyOS CRM, sales, onboarding, projects, requirements, design, development, QA, finance, client portal, handover, customer success, maintenance, AI agents and governance into one consistent system of record.

1. Domain Model Principles

- AgencyOS must have one canonical source of truth for each business concept.
- Modules may have their own tables/services, but cross-module references use stable IDs and explicit relationships.
- Every tenant-owned record is organization-scoped.
- Sensitive records require explicit authorization.
- State transitions are controlled and auditable.
- Financial, approval and deployment histories are not silently overwritten.
- Versions are first-class records for scope, quotes, invoices, designs, prototypes, builds, releases and policies where required.
- Events describe important domain transitions; they do not replace authoritative state.

2. Core Tenant Hierarchy

PLATFORM → ORGANIZATION → USERS/MEMBERS → CLIENTS → PROJECTS → PROJECT RESOURCES.

- Organization is the primary tenant boundary.
- Projects belong to exactly one organization.
- Client records are organization-scoped.
- Users may belong to an organization through membership records.
- Project membership determines project-level access.

3. Core Entity Groups

- Identity: User, Organization, Membership, Role, Permission, Session, API Credential.
- CRM: Lead, Contact, Opportunity, Sales Activity, Quote.
- Client: Client Account, Client Contact, Portal User, Approval.
- Project: Project, Project Member, Milestone, Task, Dependency, Project Event.
- Scope: Requirement, Feature, Scope Version, Change Request, Change Version.
- Design: Design Project, Design Version, Screen, Component, Prototype, Prototype Version.
- Development: Development Task, Repository, Branch/Worktree, Commit, Build, Artifact.
- QA: Test Suite, Test Case, Test Run, Defect, Readiness Assessment.
- Finance: Invoice, Invoice Line, Payment, Payment Submission, Receipt, Refund, Reconciliation, Payment Account.
- Communication: Conversation, Message, Channel, WhatsApp Group/Link, Notification.
- AI: Agent, Agent Task, Model Provider, Model, Routing Policy, Agent Run, Tool Call, Memory Record.
- Handover/Success: Handover Package, Completion Record, Maintenance Plan, Support Ticket, Renewal, Customer Health, Opportunity.
- Governance: Audit Event, Policy, Approval Request, Security Event, Retention Policy, Backup Record.

4. Base Fields for Tenant-Owned Tables

- id — stable unique identifier.
- organization_id — tenant boundary where applicable.
- created_at / updated_at.
- created_by / updated_by where applicable.
- status.

- version or current_version_id where versioned.
- deleted_at only where soft-delete is appropriate.
- metadata JSON only for non-critical extensible attributes; core business fields remain typed columns.
- Do not use free-form metadata as a substitute for relational integrity.

5. Organization

- Represents an AgencyOS tenant/agency/business entity.
- Fields: id, name, status, settings, billing profile, tax profile, created_at.
- Relations: users through memberships, clients, projects, finance records, policies, agents, provider configuration.
- Invariant: organization cannot access another organization's tenant-owned records.

6. User & Membership

- User represents identity.
- Membership connects User to Organization.
- Membership contains role/status and effective dates where required.
- Project membership separately controls project access.
- Invariant: deactivated membership cannot authorize ordinary tenant actions.

7. Roles & Permissions

- Role has stable ID, name and scope.
- Permission represents an allowed action/resource combination.
- Role-permission mapping grants permissions.
- User/membership role assignment is auditable.
- Invariant: undefined actions are denied by default.

8. Client Domain

- Client Account belongs to Organization.
- Client Contact belongs to Client Account.
- Portal User maps to a User identity and authorized client/project scope.
- Client can have multiple projects.
- Client data is isolated from other clients.

9. Lead & CRM Domain

- Lead belongs to Organization.
- Lead may reference source/campaign/channel.
- Lead can link to one or more contacts.
- Opportunity represents a qualified commercial path.
- Sales activities reference lead/opportunity/contact.
- Conversion links opportunity to client/project without deleting the lead history.

10. Quote Domain

- Quote belongs to Organization and may reference Opportunity, Client and Project.
- Quote Version contains immutable commercial content.

- Quote Line belongs to Quote Version.
- Quote acceptance references exact version.
- Accepted quote becomes the commercial baseline for downstream project/finance workflows.
- Invariant: accepted historical quote version cannot be silently mutated.

11. Project Domain

- Project belongs to exactly one Organization and Client.
- Project may reference originating Opportunity/Quote.
- Project has owner/PM and members.
- Project contains milestones, tasks, requirements, changes, designs, builds, QA, invoices and handover records.
- Project has lifecycle state.
- Invariant: project resources cannot cross tenant boundaries.

12. Project Lifecycle States

ONBOARDING → ACTIVE → BLOCKED / WAITING_FOR_CLIENT / WAITING_FOR_PAYMENT
 → FINAL_QA → READY_FOR_PRODUCTION → DEPLOYING → PRODUCTION_VALIDATED →
 HANDOVER_READY → HANDOVER_IN_PROGRESS → COMPLETED →
 MAINTENANCE/ARCHIVED.

- Cancellation and exception states are controlled by policy.
- State transitions generate domain events.

13. Requirement & Scope Domain

- Requirement belongs to Project.
- Feature may group requirements.
- Scope Version is an immutable snapshot of included/excluded requirements.
- Change Request references a current scope version.
- Change Version records proposed/approved change impact.
- Invariant: client comments cannot silently mutate an approved scope version.

14. Requirement States

DRAFT → CLARIFICATION → PROPOSED → APPROVED → FROZEN → CHANGED/RETIRED.

- Only authorized roles can approve/freeze scope.
- Frozen scope requires Change Request for material modifications.

15. Change Request Domain

- Change Request references project and originating scope/version.
- Contains requested change, reason, classification, impact, price/timeline impact, status and approvals.

Status: SUBMITTED → ANALYSIS → QUOTED/INCLUDED → PENDING_APPROVAL →
 APPROVED/REJECTED → IMPLEMENTED → CLOSED.

- Approved changes create a new scope version where required.

16. Design Domain

- Design Project belongs to Project.
- Screen belongs to Design Project.
- Design Version is immutable.
- Prototype references a Design Version.
- Prototype Version references exact build/source.
- Client approval references exact design/prototype version.
- Invariant: approved design baseline cannot be overwritten.

17. Development Domain

- Development Task references Project/Requirement/Feature.
- Repository belongs to Organization/project context.
- Commit references repository.
- Build references commit.
- Artifact references build.
- Release references approved artifact/build.
- Invariant: production release must point to an identifiable artifact.

18. Development Task States

BACKLOG → READY → IN_PROGRESS → BLOCKED → REVIEW → TESTING → DONE.

- Only evidence-backed transitions can reach DONE.
- Dependencies prevent invalid progression.
- Task completion does not automatically mean project completion.

19. QA Domain

- Test Suite groups Test Cases.
- Test Run references build/release.
- Test Result references Test Case and Run.
- Defect references test result/build/project.
- Readiness Assessment summarizes configured production gates.
- Invariant: a release-blocking defect prevents production readiness unless authorized exception exists.

20. Defect States

OPEN → TRIAGED → IN_PROGRESS → FIXED → RETEST → VERIFIED → CLOSED.

- Alternative: REOPENED.
- Severity and priority are separate fields.
- Defect remains linked to affected build/release.

21. Finance Domain

- Invoice belongs to Organization and Client/Project.
- Invoice Version or amendment history preserves issued state.
- Invoice Line belongs to invoice.
- Payment Submission records a client's claimed payment.

- Payment is the verified financial transaction.
- Receipt references verified Payment.
- Reconciliation links payments to authoritative account/statement records.
- Refund references original Payment.
- Invariant: unverified payment cannot be treated as verified financial completion.

22. Finance State Relationships

Invoice: DRAFT → ISSUED → PARTIALLY_PAID → PAID / OVERDUE / VOID / CANCELLED.

Payment Submission: PENDING_VERIFICATION → VERIFIED / REJECTED / DUPLICATE / UNMATCHED.

- Payment verification updates invoice balance and milestone financial state.
- Financial state must never depend only on a chat message.

23. Milestone Domain

- Milestone belongs to Project.
- Milestone can have operational status and financial status separately.
- Milestone may reference invoice/payment requirements.
- Payment gate controls downstream workflow where policy requires.
- Invariant: operational completion does not automatically imply payment verification.

24. Communication Domain

- Conversation belongs to organization/project/client context.
- Message belongs to Conversation.
- Channel identifies WhatsApp, portal, email or internal communication.
- Formal approvals/payments/scope changes link messages to structured records.
- Invariant: conversational text cannot silently mutate authoritative commercial/financial state.

25. WhatsApp Domain

- Lead WhatsApp conversation may map to Lead/Contact.
- Client WhatsApp group may map to Project.
- AgencyOS internal group maps to Organization/Project staff context.
- Messages carry sender, channel, timestamp and optional structured reference.
- Group membership changes are auditable.

26. AI Agent Domain

- Agent defines role/capabilities.
- Agent Task assigns work to an Agent within organization/project context.
- Agent Run records execution.
- Tool Call records privileged tool usage.
- Model Provider and Model identify execution engine.
- Routing Policy determines model selection.
- Agent identity is separate from human User identity.

- Invariant: model selection never bypasses agent permissions.

27. AI Memory Domain

- Memory Record belongs to an explicit scope: organization, client, project, agent or task.
- Memory has type, source, created_at, retention and access policy.
- Memory references source records where possible.
- Invariant: retrieval must apply the same authorization scope as the underlying data.

28. AI Routing Domain

- Model Provider has provider identity/configuration status.
- Model represents available model/version.
- Routing Policy maps agent/task type to preferred model, fallback, cost/quality constraints.
- Provider Credential is stored separately and securely.
- Agent Run records selected provider/model and policy version.

29. Approval Domain

- Approval Request references a specific resource/version.
- Approver identity and decision are stored.

Approval states: PENDING → APPROVED / REJECTED / EXPIRED / CANCELLED.

- Invariant: approval cannot be detached from the exact artifact/commercial version it approved.

30. Handover Domain

- Handover Package belongs to Project.
- Contains deliverable references, access-transfer records, documentation, release/build and known limitations.
- Completion Record references handover, final release, financial state and client acceptance.
- Invariant: project cannot enter COMPLETED without required completion gates.

31. Customer Success Domain

- Maintenance Plan belongs to Client/Project where applicable.
- Support Ticket references project/maintenance.
- Renewal references maintenance plan/contract.
- Customer Health stores explainable signals/status.
- Customer Opportunity stores expansion/repeat-project opportunity.
- Completed project remains historical while new work is separately represented.

32. Event Model

- Domain events capture meaningful state transitions.
- Examples: LeadCreated, QuoteAccepted, ScopeApproved, ChangeRequestApproved, DesignApproved, PrototypeApproved, BuildCreated, TestRunCompleted, PaymentVerified, MilestonePaid, DeploymentSucceeded, HandoverCompleted, ProjectCompleted, RenewalDue.
- Events include event_id, event_type, aggregate_type/id, organization_id, occurred_at, actor, payload_version and correlation_id.
- Events should be idempotently processed.

33. Event vs State

- State is the current authoritative condition.
- Event records what happened.
- Do not reconstruct critical financial/security state solely from unreliable event delivery.
- Use transactional writes plus an outbox/event mechanism where asynchronous propagation is required.

34. Core Invariants

- Every tenant-owned record belongs to exactly one valid organization.
- Foreign keys cannot point to missing parent records.
- Project/client relationships cannot cross organizations.
- Approved versions are immutable.
- Only authorized roles can perform protected transitions.
- Payment verification requires evidence.
- Receipt requires verified payment according to policy.
- Production release references an approved artifact.
- Completion requires configured gates.
- Deleted/archived records cannot silently reappear as active.
- Event processing is idempotent.
- Critical identifiers are stable.

35. Versioning Strategy

- Use immutable version records for business artifacts whose history matters.
- Examples: Quote Version, Scope Version, Design Version, Prototype Version, Policy Version, Release/Build Version.
- Each version has version_number, parent/version lineage, status, created_by, created_at and content/hash where appropriate.
- Current version is referenced by the parent record.
- Never overwrite historical approved content.

36. Optimistic Concurrency

- Use version/revision numbers or equivalent concurrency control for frequently edited records.
- Reject stale updates.
- Prevent two agents from silently overwriting each other's work.
- Surface conflicts for resolution.

37. Idempotency

- External webhook events use idempotency keys.
- Payment submissions cannot create duplicate verified transactions.
- Deployment callbacks cannot repeat completion side effects.
- Notifications should have deduplication where needed.
- Agent retries must not create duplicate business records.

38. Soft Delete / Archive

- Use soft-delete only where recovery/history requires it.
- Financial/audit records should not be casually deleted.
- Archived projects remain historically searchable to authorized users.
- Deletion must follow retention policy.
- Foreign-key behavior must be explicit.

39. Audit Data Model

- Audit Event references actor, organization, project/resource, action, outcome and correlation ID.
- Audit logs are append-oriented.
- Sensitive values are redacted.
- Audit access is itself controlled and audited.

40. Database Integrity & Constraints

- Use foreign keys for core relationships.
- Use unique constraints for business identifiers.
- Use check constraints for valid states where practical.
- Use transactional boundaries for coupled state changes.
- Use indexes for tenant_id, project_id, status, timestamps and high-value lookup keys.
- Do not rely on application code alone for critical uniqueness/integrity.

41. Recommended Core Tables

- organizations, users, memberships, roles, permissions, role_permissions.
- clients, client_contacts, portal_users, project_members.
- leads, opportunities, sales_activities, quotes, quote_versions, quote_lines.
- projects, milestones, tasks, task_dependencies, project_events.
- requirements, features, scope_versions, scope_items, change_requests, change_versions.
- design_projects, screens, design_versions, prototypes, prototype_versions.
- repositories, branches, commits, builds, artifacts, releases, development_tasks.
- test_suites, test_cases, test_runs, test_results, defects, readiness_assessments.
- invoices, invoice_lines, payments, payment_submissions, receipts, refunds, reconciliation_records, payment_accounts.
- conversations, messages, channels, notifications.
- agents, agent_tasks, agent_runs, tool_calls, model_providers, models, routing_policies, provider_credentials, memory_records.
- approvals, handover_packages, handover_items, completion_records.
- maintenance_plans, support_tickets, renewals, customer_health, customer_opportunities.
- audit_events, policies, policy_versions, security_events, retention_policies, backup_records.

42. Indexing Strategy

- Tenant-scoped indexes: organization_id + common filters.
- Project-scoped indexes: project_id + status/timestamps.
- Client-scoped indexes.
- Foreign-key indexes.

- Unique business identifiers.
- Event correlation and aggregate indexes.
- Financial lookup indexes: invoice number, payment reference, status.
- AI run indexes: agent/project/time.
- Audit indexes: organization/resource/time.

43. Data Consistency Boundaries

- Strong consistency for permissions, financial balances, approvals and state transitions.
- Eventual consistency may be used for notifications, analytics, search indexes and non-critical dashboards.
- Client portal may cache data only with a defined freshness policy.
- Never use eventually consistent analytics as the source of truth for financial authorization.

44. Migration Strategy

- All schema changes are version-controlled.
- Migration is reversible where practical.
- Production migration is tested against representative data.
- Destructive changes require explicit approval.
- Backfill jobs are idempotent.
- Application and database rollout compatibility is considered.

45. Seed & Reference Data

- Roles/permissions.
- Supported project states.
- Payment states.
- Defect severities.
- Agent types.
- Model/provider catalog.
- Policy defaults.
- Notification types.
- Reference data is versioned and environment-safe.

46. Data Access Patterns

- Service/repository layer should enforce organization context.
- Do not pass raw unrestricted database access to agents.
- Use domain services for sensitive transitions.
- Use read models for dashboards where appropriate.
- Use transactions for coupled changes.
- Use background jobs for long-running tasks.

47. Database Security Invariants

- RLS/authorization applies to all tenant data paths.
- Privileged service accounts are minimized.
- Database credentials are secret-managed.

- Backups are encrypted/protected.
- Sensitive columns are protected according to policy.
- Database audit access is restricted.

48. Definition of Done

- Canonical entities are defined.
- Relationships and foreign keys are defined.
- All major lifecycle states are defined.
- Critical state transitions are constrained.
- Domain events are defined.
- Core invariants are documented and tested.
- Versioned artifacts are immutable.
- Tenant isolation is enforced.
- Financial records are auditable.
- AI memory/context is scoped.
- Agent runs and tool calls are traceable.
- Audit model exists.
- Indexes support major workflows.
- Migrations are version-controlled.
- Concurrency/idempotency rules exist.
- Archive/deletion rules are defined.
- Database model supports the complete AgencyOS lifecycle.

49. Domain Relationship Summary

Parent	Child / Related	Relationship
Organization	Users/Memberships	1 → many
Organization	Clients	1 → many
Client	Projects	1 → many
Project	Milestones / Tasks / Requirements	1 → many
Project	Scope Versions / Change Requests	1 → many
Project	Design / Prototype / Builds / QA	1 → many
Project	Invoices / Payments / Handover	1 → many
Quote	Quote Versions	1 → many
Scope Version	Requirements/Scope Items	1 → many
Design Version	Screens	1 → many
Commit	Builds	1 → many
Build	Artifacts / Test Runs	1 → many
Invoice	Payments	1 → many
Payment	Receipt / Refund	1 → many
Agent	Agent Tasks / Runs	1 → many
Agent Run	Tool Calls	1 → many

Client	Maintenance / Renewals / Opportunities	1 → many
--------	--	----------

50. Master Domain Flow

ORGANIZATION → CLIENT/LEAD → OPPORTUNITY → QUOTE VERSION → PROJECT →
SCOPE VERSION → DESIGN VERSION → PROTOTYPE VERSION → DEVELOPMENT TASK →
COMMIT/BUILD → QA/RELEASE → INVOICE → PAYMENT/RECONCILIATION → PRODUCTION
→ HANDOVER → COMPLETION → MAINTENANCE → RENEWAL/OPPORTUNITY

Cross-cutting all domains: IDENTITY + PERMISSIONS + POLICY + AI ORCHESTRATION +
MEMORY + EVENTS + AUDIT + VERSIONING + SECURITY.